

runtime_confirmation_20260822

BOB-168 — runtime confirmation, RED/GREEN, mutations, negative control

Revision: 2 **Last modified:** 2026-08-22T10:47:10Z **Evidence class:** RUNTIME (§11.4.226) — the real aggregator executed, exit codes observed. Supersedes the source-class scoping of `runner_behaviour_20260821.md`, whose analysis it confirms.

Mandate: §11.4.224 (test-first), §11.4.115(F)/(G), §11.4.201(1) (11), §1.1 (paired mutation), §12.6 / 30-40% host cap.

How this was confirmed without firing the bank

The prior round left runtime confirmation OWED for a real reason: the bank contains DDoS-class and sustained-load challenges, siblings were live, and firing it would breach the host cap. That constraint still held. The runner also exposes **no filter, no dry-run, and no entry-selection env var** — its `CHALLENGES_DIR` is derived from the script's own location and its roster is a hardcoded array. That absence is itself worth reporting: there is no supported way to exercise one path of this aggregator in isolation.

But the missing-entry path needs no challenge to run — by definition it is the path taken when nothing *can* run. So the real script was executed in a directory tree where the challenges submodule is absent. **Zero challenges executed; the cost was one shell process.**

This is not the replica the previous round correctly refused to build. The artifact under test is the real file, and byte-identity was asserted before running it:

```
real      :
6e4a9c1f0cef119e2c1924a08a6e08b3b71cb4a4e9ecf8c1df9ec068a4e9c0bf
under-test:
6e4a9c1f0cef119e2c1924a08a6e08b3b71cb4a4e9ecf8c1df9ec068a4e9c0bf
IDENTICAL – this is the real runner, not a replica
```

Only the *environment* differed, and it differed toward a real, reachable state. §11.4.115(G) precondition provenance: **OBSERVED-reachable, not constructed** — a git clone without --recursive produces exactly this, so this is the defect's natural habitat rather than a hand-built precondition.

The defect, observed (pre-fix)

```
=== Boba Challenge Aggregator ===
  SKIP: no_suspend_calls_challenge.sh – not found
  SKIP: host_no_auto_suspend_challenge.sh – not found
  ... (14 more) ...
  SKIP: challenges_describe_challenge.sh – not found or not
executable
=== Summary ===
PASS: 0
FAIL: 0
SKIP: 16
TOTAL: 16
```

```
##### OBSERVED EXIT CODE: 0 #####
```

Sixteen challenges named, **zero executed, exit 0**. This is materially worse than the defect as filed. The filed concern was one dangling entry tolerated forever; the actual reachable behaviour is that an **entirely un-run challenge bank reports success** — the un-run bank and the fully-passing bank are indistinguishable to the exit code.

RED (§11.4.224 — test written and observed failing first)

tests/unit/test_run_all_challenges_missing_entry.sh, against the unmodified runner:

```
  FAIL: all entries absent -> expected exit 2, got 0 (pre-fix
behaviour: 0)
  FAIL: all entries absent -> expected 'MISSING: 16' in summary
  PASS: NEGATIVE CONTROL: complete roster, all passing -> exit 0
  FAIL: NEGATIVE CONTROL: complete roster -> expected 'MISSING: 0'
  FAIL: one entry not executable -> expected exit 2, got 0
  FAIL: one entry not executable -> expected 'MISSING: 1'
  PASS: a real challenge failure -> exit 1 (unchanged, outranks
```

MISSING)

PASS: fixture roster is in sync with the runner's
CHALLENGE_SCRIPTS
RESULT: 3 passed, 5 failed

The two invariants that must **not** change — a healthy roster exits 0, and a real failure exits 1 — already passed here, before any edit. That is deliberate: it pins them as pre-existing behaviour the fix must preserve, not as new behaviour the fix creates.

GREEN (post-fix)

PASS: all entries absent -> exit 2 (bank could not be run)
PASS: all entries absent -> MISSING: 16 counted separately from
SKIP
PASS: NEGATIVE CONTROL: complete roster, all passing -> exit 0
PASS: NEGATIVE CONTROL: complete roster -> MISSING: 0
PASS: one entry not executable -> exit 2
PASS: one entry not executable -> MISSING: 1
PASS: a real challenge failure -> exit 1 (unchanged, outranks
MISSING)
PASS: fixture roster is in sync with the runner's
CHALLENGE_SCRIPTS
RESULT: 8 passed, 0 failed

§1.1 paired mutations — every assertion proved load-bearing

#	mutation	expected	observed
M1	MISSING -gt 0 -> -gt 999999 (revert to pre-fix: MISSING never blocks)	the two absent/exit-2 assertions fail	2 failed
M2	MISSING -gt 0 -> -ge 0 (block even at zero)	the NEGATIVE CONTROL fails	1 failed
M3	delete the MISSING: n summary line	the three count assertions fail	3 failed
M4	add a phantom name to the runner's roster	the roster-drift guard fails	5 failed

#	mutation	expected	observed
M5	move the MISSING exit-2 block ABOVE the FAIL exit-1 block (reviewer-authored)	case 4b fails	1 failed (survived 8/8 before case 4b existed)
B-1	force make_root to exit 1 (fixture-setup failure)	every case fails loudly, nothing executes	all fail, RUN: count 0

M2 is the one that matters most: it proves the negative control has teeth. A fix that reddened a healthy bank would be caught, so “exit 2 on MISSING” is not achieved by making the runner red always (§11.4.201(1)).

M1 is the canonical §11.4.115(F) mutation — it restores the pre-fix exit semantics and the guard immediately reproduces the original defect.

Restoration verified by sha256 after each mutation; a §11.4.84 residue scan for 999999 / phantom_entry / MUTAT over the runner returns clean.

shellcheck: clean for the test; for the runner the accurate statement is **no new findings** — it emits a pre-existing info-level SC1091 (a sourced path shellcheck cannot follow) that predates this change and is untouched by it.

Review round — findings addressed (2026-08-22)

Independent review returned NO-GO with 1 BLOCKING + 2 IMPORTANT + 4 MINOR against the test and the evidence, not against the core change. All are closed.

B-1 (BLOCKING) — the fixture’s own failure path failed OPEN, and could have executed the real bank. Reproduced here link-by-link on bash 5.2.37 before fixing (§11.4.6):

```
subshell exit 1 inside R="$$(make_root ...)" -> caller SURVIVES (no
set -e)
cd "" -> rc=0, cwd UNCHANGED
combined, in a sandbox -> the aggregator ran
from the
INVOKER's cwd
```

From the project root that is the real aggregator with the real submodule — `ddos_health_flood`, `stress_sustained_load`, `chaos_failure_injection` — against the 30-40% host cap with a sibling stream live. The anti-replica check whose whole purpose is to catch a mid-run edit (the BOB-068 shared-checkout reality) would have *triggered* the outcome it exists to prevent. §11.4.252 fail-open on a dangerous combination.

Fixed by making `run_runner` and `populate_all` fail CLOSED: an empty or non-conforming root yields RC=97 and a loud assertion failure with nothing executed. Verified by mutating `make_root` to `exit 1` and re-running:

```
FAIL: all entries absent -> expected exit 2, got 97
FAIL: case2 fixture setup failed
... every case FAILs loudly ...
grep -c 'RUN:' -> 0          # zero challenges executed
```

I-1 (IMPORTANT) — a reviewer mutation survived: the FAIL > MISSING precedence was unguarded. Case 4 used a complete roster, so no case ever had FAIL and MISSING set at once, and swapping the two exit blocks passed 8/8. Added case 4b (complete roster + one failing stub + one deleted entry). Re-running the reviewer's M5 against it:

```
M5 (exit-2 block moved above exit-1):
  FAIL: FAIL + MISSING together -> expected exit 1, got 2
(precedence inverted?)
  RESULT: 9 passed, 1 failed
```

The mutation that survived review is now caught. This is the §11.4.194(6)(d) lesson in its purest form: six author-written mutations all died, and the first reviewer-written mutation the author had not thought of walked straight through.

I-2 / M-1 — export twins regenerated for all four BOB-168 documents (12 files, HTML+PDF+DOCX) using the project's own `convert_file` from `scripts/generate_markdown_exports.sh`, scoped to these files rather than running the repo-wide loop (which would have churned a live sibling stream's docs). `runner_behaviour_20260821.md` gained the §11.4.44 header it never had.

On the DOCX specifically: `.gitignore:262-270` makes `*.docx` **deliberately untracked** outside two named governance files ("Research-doc DOCX exports are on-demand only"). The docs/qa convention is therefore three TRACKED formats — `.md` + `.html` + `.pdf` — with `.docx` generated on demand and correctly ignored. All four documents' tracked twins verified fresh (no `.md` newer than any sibling) and non-degenerate (each PDF 29-38 KB, %PDF magic

present). Verifying the reviewer's own measurement:
"SUPERSEDED" now returns 1 hit in
runner_behaviour_20260821.html, where it previously returned 0.

M-2 — the not-executable reason said "cannot attest anything", which overstates: the runner invokes via bash "\$path", which does not require +x, so a mode-stripped entry could technically run. Reworded to "refused by the -x gate" with the nuance in a code comment. Behaviour deliberately unchanged — refusing remains the correct conservative call, and that gate predates this change.

M-4 — accepted as recorded: sharing exit 2 with the BOBA_DURABLE helper-missing branch is deliberate, same fix-the-checkout class.

Stated honestly: what was NOT verified

- **The real repository's aggregator was never run end-to-end.** Doing so executes the DDoS and sustained-load challenges and would breach the 30-40% host cap with three sibling streams live. It remains un-run here by choice.
- Consequently, that the current checkout exits **0** is an **inference**, not an observation: all 16 entries were probed present and executable, so MISSING is 0 and the exit code is decided solely by FAIL, exactly as before this change. The change is a **no-op for a healthy checkout** and bites only when the roster is broken. The negative-control case exercises that same path with stubs and passes — but on stubs, not on the real challenges.
- **No claim is made that the 16 challenges pass.** This work touched the aggregator's bookkeeping, nothing about what the challenges assert.
- scripts/pre_build_verification.sh:1792 was observed to carry the same SKIP/MISSING conflation one level up, for the *other* runner (challenges/scripts/run_all_challenges.sh not found or not executable -> SKIP). It is **out of scope for this change** and is reported, not fixed.